Design and Implementation of a Class Scheduling System Using Answer Set Programming and Linear Programming for the College of Science, Engineering, and Architecture at Ateneo de Naga University

Jherald Owen B. Aquino
Electronics and Computer Engineering Department
Bachelor of Science in Computer Engineering
Naga City, Philippines
jhaquino@gbox.adnu.edu.ph

Keanu Dave R. Andal
Electronics and Computer Engineering Department
Bachelor of Science in Computer Engineering
Canaman, Philippines
kdandal@gbox.adnu.edu.ph

Gracielle Anthonette B. Bricia
Electronics and Computer Engineering Department
Bachelor of Science in Computer Engineering
Baao, Philippines
gbricia@gbox.adnu.edu.ph

Pio Niccolo J. Caoili
Electronics and Computer Engineering Department
Bachelor of Science in Computer Engineering
Naga City, Philippines
pncaoili@gbox.adnu.edu.ph

Engr. Celeste M. Ojeda, PhD
Electronics and Computer Engineering Department
Academic Advisor
Naga City, Philippines
cojeda@gbox.adnu.edu.ph

***Abstract*—Creating class schedules in universities is often a difficult and time-consuming task, especially when done manually. It involves assigning courses, instructors, rooms, and time slots while making sure there are no conflicts and that all requirements are met. In the College of Science, Engineering, and Architecture (CSEA) at Ateneo de Naga University, this process can lead to errors such as overlapping schedules, uneven workload distribution, and inefficient use of resources. This study presents the design and implementation of a web-based class scheduling system that uses Answer Set Programming (ASP) and Linear Programming (LP) to automate the scheduling process. The system organizes scheduling data into a structured database and applies both hard constraints (which must always be followed) and soft constraints (which improve schedule quality). It then generates possible schedules using the Clingo solver for ASP and the PuLP solver with HiGHS for LP. The performance of both methods was evaluated based on solver time, penalty points, and constraint satisfaction rate. Results showed that ASP can generate schedules faster, while LP produces schedules with fewer violations of soft constraints. Compared to manual scheduling, the automated system obtained better evaluated results in constraint satisfaction and schedule quality metrics.**

Keywords—Class Scheduling System, Answer Set Programming, Linear Programming, Clingo, PuLP

I. Introduction

Timetabling, also known as course scheduling, is the process of assigning academic events such as lectures and laboratories to specific time slots, rooms, and instructors while satisfying institutional rules and minimizing scheduling conflicts [1]. In educational institutions, effective timetabling is crucial for optimal faculty management and performance. A well-structured timetable optimizes resource allocation and minimizes scheduling conflicts. In contrast, a poorly structured timetable can create scheduling conflicts, reduce the efficient use of rooms and timeslots, and place an unnecessary burden on faculty and administrators [2].

In many institutions, such as Ateneo de Naga University (ADNU), preparing class timetables for each semester is a demanding and error-prone task. Higher education continually experiences changes such as curriculum changes, shifting student populations, and varying faculty availability and

resources, all of which require better resource allocation and planning. However, manual timetabling complicates the process, as it is ineffective and labor-intensive. Manual timetabling relies on previous semesters as a template for creating future schedules; however, students, professors, and classroom availability change over time, necessitating new decisions to be made each semester. Previous researchers have noted that manual scheduling is ineffective for handling large and dynamic datasets; this inefficiency of manual timetabling often leads to conflicts and errors, and it requires constant intervention to ensure feasibility, especially with the changing institutional requirements every semester, such as faculty load adjustments, room capacity limitations, varying class sizes, and new curricula [3].

Due to the various department laboratories, the coexistence of new and old curricula, the availability of classrooms, and the faculty who teach a variety of courses across departments, creating a schedule within the College of Science, Engineering, and Architecture (CSEA) is challenging. Automated class scheduling systems have been explored to address this challenge in higher education due to their efficient resource utilization. These systems are known to be effective in allocating resources such as classrooms, instructors, students, laboratories, and venues. Class scheduling is recognized as an NP-hard combinatorial optimization problem. This classification means that finding an optimal solution is computationally challenging as no known polynomial-time algorithm can guarantee an optimal timetable, the time to find optimal solutions grows exponentially with the size of the problem; in this study, the problem size is the number of courses, course offerings, rooms, timeslots, instructors and instructor's time preference, making it computationally impractical to explore all timetable configurations [4]. The difficulty increases, especially since it must satisfy all hard constraints and optimize the solution for the soft constraints [5].

Hard constraints are mandatory requirements that the automated scheduler must meet for the generated schedule to be considered valid or feasible, such as preventing an instructor from teaching two classes at the same time and ensuring there are no three consecutive classes for both instructors and students. A violation of a hard constraint would render the generated schedule invalid and unusable. Meanwhile, soft constraints are flexible goals rather than mandatory conditions and may improve the quality of the schedule, such as ensuring that each instructor has the correct teaching unit per semester. These constraints are optimized only after hard constraints are satisfied. Moreover, violating a soft constraint does not invalidate the schedule, but it results in a corresponding penalty [6].

Over the years, various approaches have been explored to solve this problem, for instance, mathematical programming models, genetic algorithms, and deep reinforcement learning. However, these approaches have several flaws, such as scalability issues, suboptimal solutions, and oversimplification of real-world constraints [7][8]. In light of these limitations, previous studies have shown that Answer Set Programming (ASP) and Linear Programming (LP) are effective methods for solving class scheduling problems. ASP is a declarative programming language that excels at managing rules and logical constraints, while LP is known for its efficiency in resource management and allocation [9][10]. These programming languages have different methodologies; however, both approaches have their own advantages for addressing CSEA's scheduling needs.

The purpose of this study was to design and implement a class scheduling system utilizing ASP and LP to minimize scheduling conflict, improve resource utilization, and optimize the workload of faculty. Through the implementation of the automated scheduling system, CSEA was provided with a tool to support schedule generation, reduce scheduling conflicts, and improve the overall management of the schedule.

II. Related works

Previous studies have extensively explored automated class scheduling systems to address the manual handling of university course schedules. The development of LUNCH (Logic-based University Course Scheduler) presented a free and open-source system utilizing the CLINGO solver to automate manual class scheduling, modelling complex requirements using hard constraints (inviolable rules) and soft constraints (preferences with weights) [11]. Similarly, an AI-based tool for Curriculum- Based Course Timetabling (CB-CTT) at the University of Potsdam leveraged Answer Set Programming (ASP) and Clingo to process logic programs and minimize soft constraint violations efficiently [12]. At the University of Nebraska Omaha, the Peter Kiewit Institute utilized ASP and Clingo to strategically reassign timeslots and enhance an existing manually created schedule, prioritizing the minimization of critical conflicts and professor preference violations [13]. Furthermore, other optimization models, such as Integer Linear Programming (ILP), have shown great efficacy. Kristin Bushman's thesis addressed class timetabling challenges by introducing a web-based scheduling tool that combined visual planning with ILP using the Gurobi optimizer to handle hard constraints and balance softer goals like workload and room utilization [14]. Concurrently, Hoshino and Albers (2024) dealt with creating master timetables, batching students in cohorts using ILP with Google OR-Tools' MPSolver, efficiently assigning courses, teachers, cohorts, and rooms [15].

These prior studies established Answer Set Programming (ASP) and Linear Programming (LP) as robust, state-of-the-art methodologies for resolving NP-hard educational scheduling problems [11]–[15]. However, existing literature largely investigated these solvers in isolation or focused on partial scheduling optimizations, such as specific cohort batching or timeslot reassignments. This study synthesized these foundational methodologies by comparing them directly with one another within a unified, web-based framework. By implementing an identical and comprehensive set of hard and soft institutional constraints across both paradigms, our research built upon the successes demonstrated in previous works to conduct a comparative analysis. This allowed us to evaluate and determine the most practical and efficient solver, comparing ASP (via Clingo) and LP (via PuLP/HiGHS) in terms of solver time, constraint satisfaction rates, and penalty points.

III. METHODOLOGY

A. System's Model

The figure below shows the system model that was followed in this study.

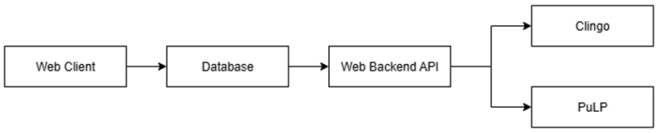


Fig. 1. System Model of the Study

The web client, functioning as the primary user interface, was developed using React.js with TypeScript, utilizing Tailwind CSS to create a responsive and modern graphical user interface. This frontend application allowed users to modify data or generate schedules. The database served as the central data repository for the class scheduling system. It stored all the important information needed to create conflict-free timetables, using the MySQL database. The web backend API, acting as the crucial communication bridge between the web client and the solvers, utilized FastAPI. This Python-based framework ensured high-performance, asynchronous data transfer, and seamless integration with the solvers. After processing, the API converted this extracted data into a format that could be interpreted by the solvers to generate schedules.

B. Technical Constraints

The ASP approach used a multi-file structure in which all constraints were defined in two program files. These constraints corresponded to specific course requirements or preferences. To facilitate knowledge base updates and maintenance, the structure was designed to remain as organized and self-contained as possible. In our notation and syntax, HCn denotes the n-th hard constraint, and SCn represents the n-th soft constraint. The ASP syntax adhered to the conventions of CLINGO, where uppercase identifiers beginning with a letter indicated variables, and lowercase identifiers denoted predicates or constants.

LP used a single model-based structure in which all variables, sets, and mathematical constraints were written together within one optimization model using PuLP. Hard constraints (HCn) were expressed as equations that had to be strictly satisfied, while soft constraints (SCn) appeared in the objective function as penalty terms that the solver minimized. Unlike ASP, which structured rules declaratively, the LP approach consolidated all scheduling conditions into one organized framework, making them easier to trace, update, and maintain. Each rule was written using standard mathematical notation, such as indexed decision variables and summation expressions, allowing PuLP to convert them into a format that HiGHS solved. This structure ensured that the LP solver evaluated all feasible schedules while satisfying room capacities, instructor availability, and other institutional scheduling policies.

a) Hard Constraints

Hard constraints are considered as the unchangeable requirements that must always be satisfied for a schedule to be regarded as valid and acceptable. Hence, the generated schedule becomes unfeasible if any of the constraints are violated [16].

The table below shows the hard constraints that were applied in the study:

TABLE I. HARD CONSTRAINTS

Variable Name	***Description***	***Points (Weight)***
HC1	*Each course offering should be assigned to one professor, one room, and one timeslot.*	*40*
HC2	*Professors must not teach two classes at the same time.*	*40*
HC3	*Rooms must not handle two classes at the same time.*	*40*
HC4	*Room capacity must be respected.*	*35*
HC5	*Room type must be respected.*	*35*
HC6	*The class must be assigned to a timeslot with the correct duration.*	*40*
HC7	*Professors must not have three consecutive classes.*	*40*
HC8	*Three consecutive class schedules are not allowed.*	*40*
HC9	*Two consecutive laboratory classes are not allowed.*	*35*
HC10	*Professors must only teach courses included in their mastery. Cross-department teaching is allowed.*	*30*
HC11	*Omitted: Courses exceeding room capacity must have two or more sections.*	*30*
HC12	*Classes are not allowed during activity periods.*	*40*
HC13	*Professors must not be scheduled in these specific timeslot/s.*	*30*
HC14	*There are courses locked to a specific professor.*	*30*
HC15	*There are courses locked to a specific room.*	*30*
HC16	*There are courses locked to a specific timeslot.*	*30*
HC17	*Classes must follow standard day-pairing patterns.*	*30*
HC18	*A section must not have two classes at the same time.*	*40*
HC19/20	*Two consecutive three-hour classes are not allowed*	*35*
CR1	*If a course has allowed rooms configured (course_rooms), it must be scheduled only in those rooms (per meeting type).*	*35*

HC11 was omitted from the solver constraints because the final scheduling workflow no longer performs automatic section splitting. Instead, oversized offerings are treated as input-data preparation concerns and are scheduled as provided by the dataset or department. Although HC11 is no longer enforced, its identifier was retained in the system as a deprecated rule code rather than renumbering the remaining hard constraints. This decision preserves traceability across historical solver runs, violation logs, seed data, frontend displays, backend validation logic, and experiment documentation. Rule codes function as stable identifiers in the implementation, not merely presentation labels; therefore, renaming HC12 onward would risk changing the meaning of previously recorded results and breaking references between stored violations and their rule definitions.

The numbering intentionally contains a gap after HC10. The active hard constraints continue with their original identifiers, while HC11 is documented as deprecated/disabled. This follows a versioning approach in which removed constraints are marked inactive instead of reusing or shifting identifiers.

b) Soft Constraints

Soft Constraints are represented as the "preferences" or desired conditions that can affect the quality of the generated schedule. However, these factors can be waived if such constraints conflict with the aforementioned hard constraints [16].

The table below shows the soft constraints that were applied in the study:

TABLE II. SOFT CONSTRAINTS

Variable Name	Description	Points (Weight)
SC1	The preferred timeslots of professors should be respected.	10
SC2	Professors should not exceed a total of 27 teaching units per semester.	15
SC3	Custodian availability should be respected, but may allow exceptions.	10
SC4	First-year and second-year classes may be avoided from being scheduled at or after 18:00 (6:00 pm).	10
SC5	Prefer breaks (class–free–class) for the same professor on the same day, but penalize only when the middle (idle) slot duration exceeds 90 minutes.	10
SC6	Prefer each professor to teach on no more than 4 days per week, and prefer those teaching days to be consecutive (clustered into a continuous block).	10
SC7	Discourage classes scheduled on Saturday.	10
CR2	If a course has preferred rooms configured (course_rooms.is_preferred), the schedule should use those rooms when possible.	5

C. Solver Time

Solver Time was the total computational time of each solver to generate a schedule. It accounted for the solver's efficiency and performance; the shorter the time, the better the computational efficiency.

D. Point Metric System

To assess the performance of manual and automated class scheduling approaches, a point-based metric system was created. This system evaluated timetable quality based on adherence to hard and soft constraints, with weighted values reflecting the relative importance of each constraint. The total score indicated how well a generated or manual schedule complied with institutional requirements. Higher scores indicated that the schedule violated more constraints, resulting in poorer overall quality.

Penalty Points (P), a metric used in the assessment, will be used to gauge the quality of the generated or manual schedule. The framework calculated the total number of constraint violations found within the timetable and is calculated as follows:

$$P = \sum_{n=1}^{K} Wn * Cn \quad (1)$$

Where K represented the number of constraints considered, and Cn indicated the number of times the n-th constraint was violated. Wn denotes the weight or importance value of the n-th constraint. Every violation generates points equivalent to the weight of the constraint, indicating the severity or importance of the violation. When a high-weight constraint was violated, the total point value increased significantly, indicating a more severe scheduling flaw. Lower-weight constraints, on the other hand, contributed fewer points and had less impact on overall feasibility.

E. Constraint Satisfaction Rate

Constraint Satisfaction Rate (CSR) was used to quantify the effectiveness of the solver in satisfying both the hard and soft constraints. CSR was computed in two stages. First, a base score was derived from the point metric system by comparing the total penalty points incurred with the maximum possible penalty points and the result was expressed as a percentage. Second, the base score was adjusted according to the ratio of the counted hard violations to the number of evaluated schedule entries. Thus, a higher CSR indicated better overall constraint compliance and better scheduling performance.

The CSR was expressed in the following table:

TABLE III. TABLE FOR CSR COMPUTATION

Variable	Value
M	*Maximum Penalty Points*
P	*Total Penalty Points*
Base Score Formula:	
$Base\ Score = \left(\frac{M-P}{M}\right) \times 100$	
Variable	**Value**
E	*number of evaluated schedule entries*
H	*number of counted hard violations*
Hard Constraint Compliance Factor:	
$HCCF = max(0,min(1, \frac{E-H}{E}))$	
Constraint Satisfaction Rate:	
$CSR = Base\ Score \times HCCF$	

In **Table III**, the Base Score represented the normalized penalty-based score derived from the accumulated penalty score during scheduling validation. The Hard Constraint Compliance Factor (HCCF) served as an adjustment factor that scaled the Base Score according to the hard violation density. It was computed from the number of counted hard violations relative to the number of evaluated schedule entries. The max function ensured that the value of HCCF does not fall below zero. Therefore, the Constraint Satisfaction Rate (CSR) showcased both the quality of the schedule in terms of penalty minimization and its compliance with hard scheduling requirements that can be validly evaluated.

F. Manual Upload Checker

The manual schedule checker used a two-tier validation architecture composed of a Hybrid (DB-backend) Checker and a Local Checker. During upload processing, the Hybrid Checker performed per-row and run-level validation using available database contents such as rooms, timeslots, course offerings, and professor records. When exact database matches were unavailable, fallback checks were performed. To ensure that uploaded schedules could still be evaluated even under incomplete or historically different data, a Local Checker independently re-evaluated the upload using the CSV rows together with minimal reference data. This local checker served as the final basis of evaluation, including hard and soft violations counts, penalty points, and other summary metrics.

a) Adjusted Constraint

This section presents the adjusted constraint settings that were applied in the manual upload checker of the system. This was done to ensure that uploaded manual schedules were evaluated accurately and were not penalized based on metadata that exists only in the current academic offerings.

HC10 (Professor Mastery) constraint was always skipped during manual uploads since manual evaluation does not penalize schedules using current-offering mastery metadata. Likewise, HC14 (Professor Lock Enforcement) was also always skipped, since manual uploads are not evaluated using present-day professor-lock configurations from the current offering records.

For HC2 (Professor Overlap), the system bypasses standard ID-based overlap validation for unresolved faculty members in the database, using name-based checks as a fallback to ensure evaluations proceed without direct matches.

Several constraints related to offerings are only validated when an exact offering match is found in the database. These constraints include HC15 (Offering Room/Timeslot Locks) and HC16 (Course Timeslot Requirement). Additionally, CR1 (Allowed Room Policy) and CR2 (Preferred Room Policy) were introduced as newly added constraints that were not part of the primary constraints set in the system. However, if no match is found, these constraints are not applied to prevent previous schedules from being unfairly penalized for missing metadata not relevant to their original implementation.

Moreover, the professor's preferred timeslot was set to align with the department chairpersons for the ECE-CPE Department for the periods 2022–2024 and 2024 onwards. This ensures that the time preferences of the corresponding professors are properly evaluated as well.

b) Similarity Index

The manual upload checker used in the system evaluates a similarity-index validation comparing them to the record stored in the database. These records were from the curriculum (mixed by old and new) from each program, and the most recent offering records available from Academic Year 2025–2026, First and Second Semesters. If discrepancies arise, such as differences in course code or title, the checker reconstructs the inconsistent data to ensure that partially mismatched manual schedules can still be evaluated properly by the system. A lower similarity index does not automatically invalidate a manual schedule, but it indicates that more reconstruction was required, which may reduce direct comparability with current database-backed schedules.

G. System Usability Scale

The System Usability Scale (SUS) was developed by John Brooke in 1996 as a standardized instrument for evaluating the perceived usability of systems. The questionnaire consists of ten questions, alternating between positive and negative statements, that are scored on a five-point Likert scale. For computation, the scale would yield usability scores ranging from 0 to 100 [17].

To compute the score, the user's responses are computed based on the item number. For odd numbers, the response is subtracted by one, and for even numbers, the response is subtracted from five. This process standardizes scores to a 0-4 scale. After summing all ten adjusted responses for a total of 40, multiply by 2.5 to convert to a 0-100 scale, representing the overall SUS score. To interpret the results, a SUS score above 68 would be considered above average, and a score below 68 is considered below average [17].

Furthermore, **Table IV** shows the general grading scale of interpretation of the SUS scores [18].

TABLE IV. SUS GENERAL GRADING SCALE

SUS Score Range	Grade	Percentile Range
84.1 – 100	A+	96 – 100
80.8 – 84.0	A	90 – 95
78.9 – 80.7	A−	85 – 89
77.2 – 78.8	B+	80 – 84
74.1 – 77.1	B	70 – 79
72.6 – 74.0	B−	65 – 69
71.1 – 72.5	C+	60 – 64
65.0 – 71.0	C	41 – 59
62.7 – 64.9	C−	35 – 40
51.7 – 62.6	D	15 – 34
0.0 – 51.6	F	0 – 14

SUS is regarded for its dependability, simplicity of usage, and ability to provide quick quantitative insights into system usability and user satisfaction across multiple platforms. In order to provide a quantifiable measure of usability and support decisions about system improvement, this study made use of SUS to analyze the usability of a scheduling system. This was gathered through feedback from the respective chairpersons of each department within CSEA using structured questionnaires to evaluate the system's functionality and interface.

H. Likert Scale

The Likert scale, developed by Rensis Likert, is a widely used psychometric tool that is designed to measure attitudes, perceptions, and opinions by allowing respondents to indicate their level of agreement with a given statement. This is usually structured as a five-point scale ranging from “strongly disagree” to “strongly agree” that has an assigned numerical value from 1 to 5, respectively.

In this study, the mean scores for the survey questions related to this were used to interpret the general perception of the chairperson from each department. The table below shows the interpretation of results from the 5-point Likert scale [19].

TABLE V. LIKERT SCALE MEAN SCORE INTERPRETATION

Likert-Scale Description	Likert Scale	Likert-Scale Range
Strongly Disagree	1	1.00 - 1.80
Disagree	2	1.81 - 2.60
Neutral	3	2.61 - 3.40
Agree	4	3.41 - 4.20
Strongly Agree	5	4.21 - 5.00

I. Test and Evaluation

The automated class scheduling system was tested and evaluated through the comparison between the previous manual iterations of class schedules and the system-generated schedules by ASP and LP. The quality of the scheduling method was assessed using a points-based metric and constraint satisfaction rate (CSR), and for the automated scheduling approaches, solver execution time. Total penalty points measured the hard and soft constraint violations, where higher penalty points indicated more violations and lower quality schedules. Meanwhile, CSR was used to measure, in percentage form, the extent to which hard and soft constraints were satisfied. A higher CSR indicated better compliance of the schedule to the constraints. Solver execution time was also measured to evaluate the computational efficiency of ASP and LP. In addition, an end-user evaluation was conducted to assess the usability and perceived usefulness of the automated class scheduling system.

IV. RESULTS

The researchers illustrated performance data to compare Answer Set Programming (ASP) and Linear Programming (LP) across varying departmental scopes. To ensure the reliability and accuracy of the performance evaluation, each solver executed over ten runs. Runs 1–5 represented the first semester, while Runs 6–10 represented the second semester. Ten runs were conducted to account for possible variations in solver performance caused by differences in solver processing. Analyzing the outcomes across ten runs provides a stable and representative measure of solver time, violations, and penalty points, thereby enabling an accurate and consistent comparison between ASP and LP performance. Any violation of a hard constraint does not produce a valid schedule. For ASP, such a case results in an unsatisfiable model; for LP, it results in an infeasible model or a failed schedule generation run.

A. Comparative Interpretation Across Scopes

This section illustrates the performance of the Answer Set Programming (ASP) and Linear Programming (LP) solvers across two distinct dataset scopes: the 'All Departments' dataset and the 'ECE and CPE Department' dataset.

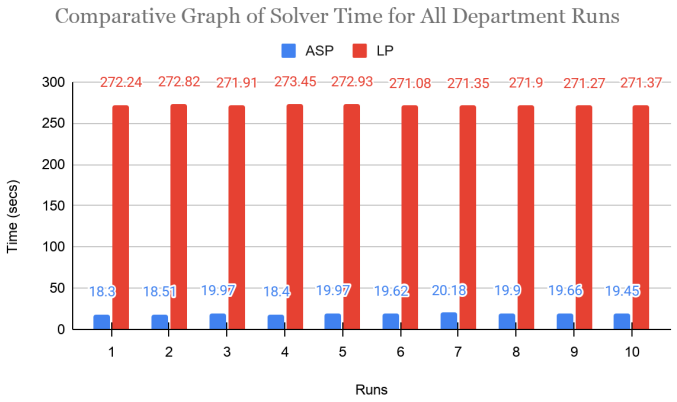


Fig. 2. Comparative Graph of Solver Time for All Department Runs

Figure 2 shows the solver execution times across all department runs. In this comprehensive scenario, ASP demonstrated substantially superior execution time performance compared to LP across large dataset instances. The average execution time for ASP was approximately 19.40 seconds (range: 18.30–20.18 seconds), whereas LP required approximately 272.03 seconds (range: 271.08–273.45 seconds). This represented a 14.03-fold speedup advantage in favor of ASP.

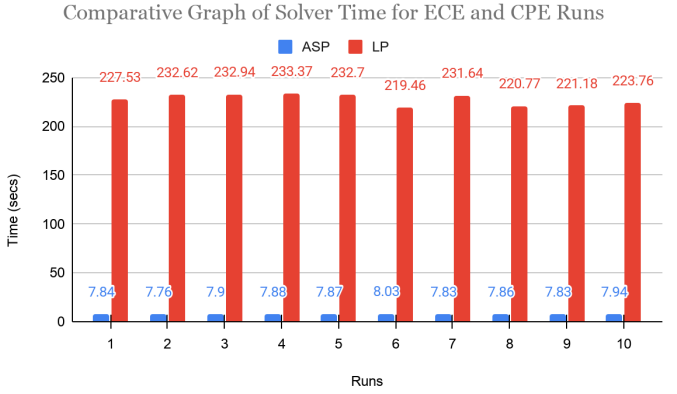


Fig. 3. Comparative Graph of Solver Time for ECE and CPE Runs

Figure 3 details the solver execution times for a reduced dataset consisting only of the ECE and CPE departments, allowing for an analysis of scalability. The solvers exhibited distinct scalability characteristics across the two scopes. ASP maintained consistently low execution times, requiring fewer than 10 seconds for the smaller dataset before scaling to approximately 19.4 seconds for the all departments dataset. In contrast, LP proved highly sensitive to data volume. Its execution time ranged between approximately 219 and 234 seconds for the ECE and CPE dataset, and increased significantly to a consistent 271–274 seconds across all departments' datasets. While ASP is exceptionally fast, it trades optimal quality for speed.

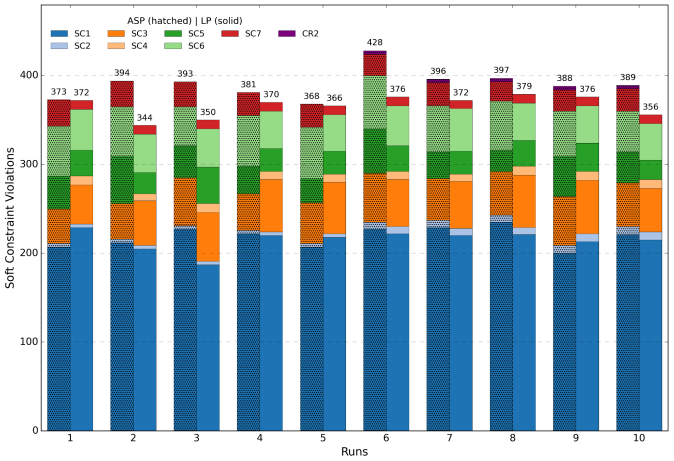


Fig. 4. Comparative Graph of Soft Constraint Violations for All Department Runs

Figure 4 presents the graph of soft constraint violations for the all-departments dataset. The results showed a higher number of violations due to the increased complexity and larger scope. In terms of both violations and corresponding penalty points, Linear Programming (LP) consistently produced fewer violations compared to Answer Set Programming (ASP). This indicated that LP is more effective in minimizing soft constraint violations even under large-scale scheduling conditions.

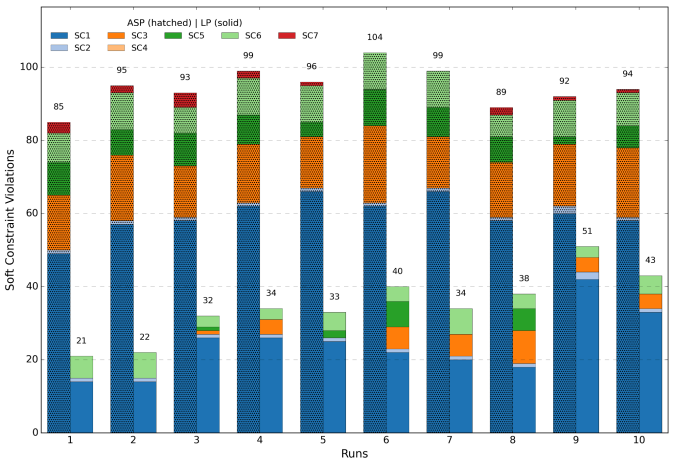


Fig. 5. Comparative Graph of Soft Constraint Violations for ECE and CPE Runs

Figure 5 illustrates the graph of soft constraint violations for the Electronics and Computer Engineering (ECE) and Computer Engineering (CPE) departments. With the reduced dataset scope, LP further improved its performance by optimizing soft constraints more effectively, resulting in fewer violations. In contrast, ASP continued to prioritize minimizing computation time rather than reducing violations. These results reinforced that LP performs better in optimizing soft constraints, particularly in more focused or smaller-scale scheduling scenarios.

Among the soft constraints, **SC1** (preferred timeslots of professors) is the most frequently violated in both figures. This indicated that accommodating faculty time preferences is the most challenging aspect of the scheduling process due to limited timeslot availability and overlapping requirements, and therefore required further optimization to improve overall schedule quality.

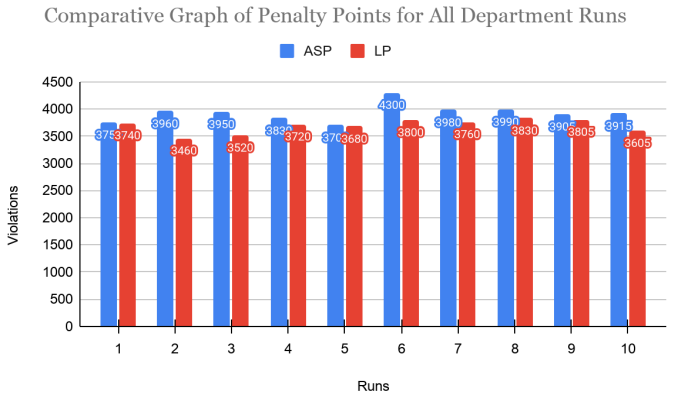


Fig. 6. Comparative Graph of Penalty Points for All Department Runs

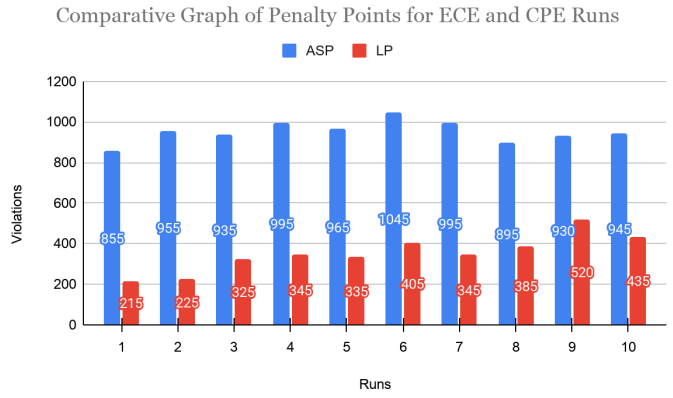


Fig. 7. Comparative Graph of Penalty Points for ECE and CPE Runs

A comparison between **Figures 6 and 7** illustrates a consistent optimization trend across varying scopes. Similar to the pattern observed in violations across both large and small departmental scopes, LP demonstrated a significant advantage over ASP in minimizing penalty points. This capability is evident in the small dataset consisting of the ECE and CPE departments (Figure 7), where LP reduced penalty points by more than half compared to ASP. Moreover, when scaling to the large dataset encompassing all departmental scopes (Figure 6), LP consistently maintained this advantage in minimizing the penalty points

.

TABLE VI. SUMMARY STATISTICS OF ASP AND LP ACROSS TEN RUNS

Scope	Solver	Mean Time (Sec)	Min–Max Time (Sec)	Mean Violations	Mean Penalty Points
All Departments	ASP	19.4	18.30 – 20.18	390.7	3928
All Departments	LP	272.03	271.08 – 273.45	366.1	3692
ECE/CPE	ASP	7.87	7.76 – 8.03	94.6	951.5
ECE/CPE	LP	227.6	219.46 – 233.37	34.8	353.5

Table VI presents the performance of Answer Set Programming (ASP) and Linear Programming (LP) across ten runs, showing their overall results together with their lowest and highest observed values.

For all departments, ASP achieved a mean time of 19.4 seconds, with values ranging from 18.30 to 20.18, whereas LP required substantially longer execution, with a mean time of 272.03 seconds and a range of 271.08 to 273.45. In terms of violations, LP produced fewer average violations (366.10) compared to ASP (390.70), and likewise obtained lower mean penalty points (3,692 vs. 3,928).

A similar pattern is observed in the ECE/CPE scope. ASP demonstrated significantly faster execution with a mean time of 7.87 seconds (7.76–8.03), while LP recorded a mean time of 227.6 seconds with a wider range of 219.46 to 233.37. However, LP also yielded better quality outcomes, with considerably fewer mean violations (34.80 vs. 94.60) and lower penalty points (353.50 vs. 951.50).

Overall, the values indicated that ASP consistently provided faster execution, while LP, despite longer and more variable runtimes, produced higher-quality schedules with fewer violations and reduced penalty points.

TABLE VII. SUMMARY OF WELCH'S INDEPENDENT SAMPLES T-TEST RESULTS FOR ASP AND LP

Metric	ASP Mean	ASP SD	LP Mean	LP SD	Mean Difference (ASP−LP)	95% CI	t(df)	p-value
Soft Constraint Violations	390.7	16.36	366.1	12	24.60	[11.03, 38.17]	3.83 (16.51)	≈ 0.0014
Penalty Points	3928	163.44	3692	125.75	236	[98.35, 373.65]	3.62 (16.89)	≈ 0.0021

Note. All tests were two-tailed with α = 0.05. Mean Difference was computed as ASP − LP. A positive mean difference indicates that ASP obtained the higher value for the metric, while a negative mean difference indicates that LP obtained the higher value. SD = standard deviation; CI = confidence interval.

Table VII presents the summary of the Welch's Independent Samples t-test comparing ASP and LP across ten runs in terms of total soft constraint violations and penalty points for all departments. The findings show that ASP

obtained significantly higher mean values for both total mean violations of 390.70 and mean penalty points of 3928.00 compared to LP (Mean = 366.10 and Mean = 3692.00, respectively). The positive mean differences indicated that LP consistently produced better outcomes with fewer violations and lower penalty scores. The computed t-values and corresponding p-values indicated that these differences are statistically significant. Overall, these findings demonstrated that LP provides better schedule quality compared to ASP, and that this advantage is consistent across all ten runs rather than attributable to chance.

B. Soft Constraint Satisfaction Results

This section illustrates how ASP and LP satisfied soft constraints across ten scheduling runs. It assessed the capability of the automated scheduling system in generating a feasible schedule that follows the institutional guidelines while improving scheduling quality through the optimization of soft constraints.

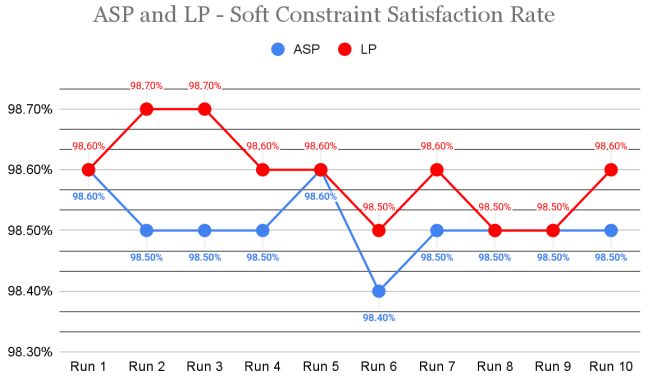


Fig. 8. Graph of ASP and LP Soft Constraints Satisfaction Rate

Figure 8 illustrated the constraint satisfaction rate of ASP and LP across ten scheduling runs. Across all the scheduling runs, all hard constraints were met by both scheduling methods, indicating that both solvers were capable of generating a feasible schedule under the given dataset and constraint set. Since hard constraints were fully satisfied in all accepted runs, the observed difference in CSR was primarily due to the optimization of soft constraints. LP accumulated lower soft penalty points, resulting in a higher constraint satisfaction rate than ASP. This suggests that LP was more effective in satisfying soft constraints under the given experimental conditions.

TABLE VIII. SUMMARY STATISTICS OF SOFT CONSTRAINT SATISFACTION RATE

Statistic	ASP	LP
Mean	98.51%	98.59%
Minimum	98.40%	98.50%
Maximum	98.60%	98.70%
Median	98.50%	98.60%

Table VIII displayed the summary statistics which further support the results, as LP obtained a higher mean CSR (98.59%) than ASP (98.51%), as well as higher minimum and maximum values. The average difference between ASP and LP was 0.08 percentage points, which indicates that LP consistently achieved slightly higher soft constraint satisfaction across all runs. Although the numerical gap was relatively small, the results demonstrated that LP provided more refined optimization of soft constraints while maintaining full compliance with hard constraints.

TABLE IX. CSR SUMMARY OF WELCH'S INDEPENDENT SAMPLES T-TEST RESULTS

Metric	ASP Mean	ASP SD	LP Mean	LP SD	Mean Difference (ASP −LP)	95% CI	t(df)	p-value
CSR%	98.51	0.06	98.59	0.07	-0.08	[-0.14, -0.02]	-2.72 (16.89)	< 0.015

Note. All tests were two-tailed with α = 0.05. Mean Difference was computed as ASP − LP. A positive mean difference indicates that ASP obtained the higher value for the metric, while a negative mean difference indicates that LP obtained the higher value. SD = standard deviation; CI = confidence interval.

Table IX contains the summary of the Welch's independent-samples t-test. It further confirmed that LP's higher mean CSR was statistically significant $t(16.89) = -2.72$, $p = 0.015$. This indicated that LP's advantage in soft constraint optimization was consistent across all runs but also statistically reliable under the study conditions. Although the difference was statistically significant, its practical magnitude was small, indicating that LP's advantage was consistent but modest.

C. Comparative Interpretation Across LP Time Limit

This section evaluates the Linear Programming (LP) solver's performance under varying time limits. By comparatively analyzing the scheduling outputs at 300-seconds, 600-seconds, and 1200-seconds, this section established a correlation between extended computational duration and schedule optimality.

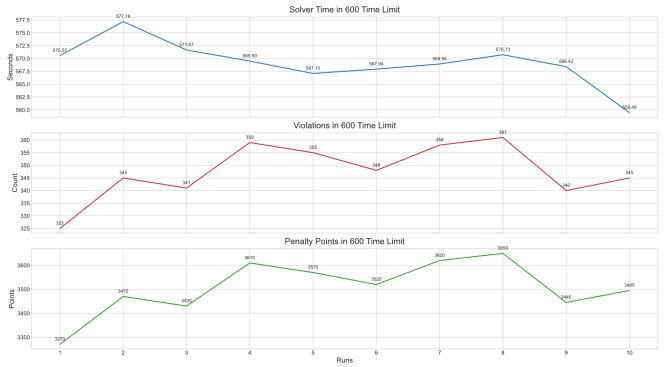


Fig. 9. Graph of LP in 600 Time Limit

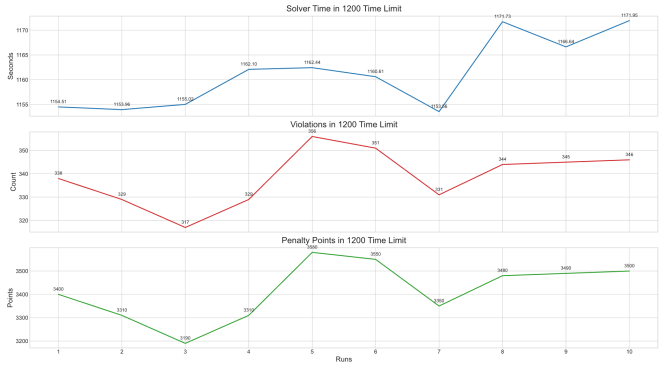


Fig. 10. Graph of LP in 1200 Time Limit

Figures 9 and 10 represent the visual progress of the LP solver when inputted time limits of 600 seconds and 1200 seconds, respectively. These graphs illustrate the solver's continuous improvement over time. Given a longer computational duration, the system actively uses the extra time to further refine the schedule, satisfying more faculty preferences and reducing conflicts compared to the shorter baseline runs.

TABLE X. STATISTICAL TIME LIMIT ANALYSIS ACROSS 10 RUNS

Time Limit	n	Mean Time (sec)	Mean Violations	Mean Penalty Points	Mean CSR
300 sec	10	272.03	366.10	3692	98.59%
600 sec	10	559.37	364	3674.5	98.60%
1200 sec	10	1158.33	351.1	3542	98.66%

Table X presents the statistical table supporting this improvement across ten test runs (n=10) for each time limit. The data reveal a clear, steady trend: as the computational time increases, both the number of scheduling violations and the total penalty points consistently decrease. Specifically, the input 300-second limit resulted in an average of 366.10 violations and 3692 penalty points. At 600 seconds, these averages dropped to 364.00 violations and 3674.50 penalty points. At the 1200-second limit, the solver further reduced violations to 351.10 and penalty points to 3542. The CSR indicated a modest upward trend with the values 98.59%, 98.60%, and 98.66%. These results suggest that allotting more computational time to LP generally improved the quality of the generated schedules under the study conditions.

D. Manual Scheduling Upload Comparison

This section compares eight (8) manually prepared schedules and automated schedules under the implemented evaluation framework, using only the constraints that could be validly verified for uploaded historical schedules.

For proper identification of the evaluated schedules, a coding format was used to indicate the specific academic year and semester in which each manual schedule was originally implemented, as shown in **Table XI**. This serves as a reference throughout the succeeding discussions and tables. Furthermore, the similarity index for each manual schedule is also provided. It can be noted that the similarity values vary widely; therefore, the comparability of the schedules was not directly identical with current database-backed schedules.

TABLE XI. MANUAL SCHEDULE REFERENCE

SCHEDULE CODE	ACADEMIC YEAR	SEMESTER	TOTAL ENTRIES	SIMILARITY INDEX
22231s	2022-2023	First Semester	192	17.19%
22232s	2022-2023	Second Semester	200	28.00%
23241s	2023-2024	First Semester	188	33.51%
23242s	2023-2024	Second Semester	184	39.67%
24251s	2024-2025	First Semester	198	48.48%
24252s	2024-2025	Second Semester	208	63.38%
25261s	2025-2026	First Semester	207	81.64%
25262s	2025-2026	Second Semester	212	71.70%

In the succeeding sections, a comparison is presented showing the number of violations, total penalty points, and CSR values of the manually created schedules from previous semesters versus the generated schedules. For manual schedules, evaluation was performed using a hybrid validation process that combined a shared validator with a local manual checker. This method enabled the assessment of uploaded previous schedules based on available file information and class time intervals, even when complete database data was not accessible. However, only selected constraints verifiable from the uploaded schedules and supporting data were considered in the manual-upload comparison.

The points and percentages obtained from the two methods, ASP and LP, were based on the mean score values from the ten runs for each solver across all departments. While individual results from all manual schedules are presented, the eight manual schedules were also averaged and labeled as MANUAL in the charts for fair comparison.

a) Hard and Soft Constraints Violations

This section provides the number of hard and soft constraints that were violated during the testing phase of this study. The comparison includes the manually created schedules from previous semesters and the automated schedules generated using the ASP and LP solvers. The following figure shows the number of soft and hard constraints that were violated and their sum.

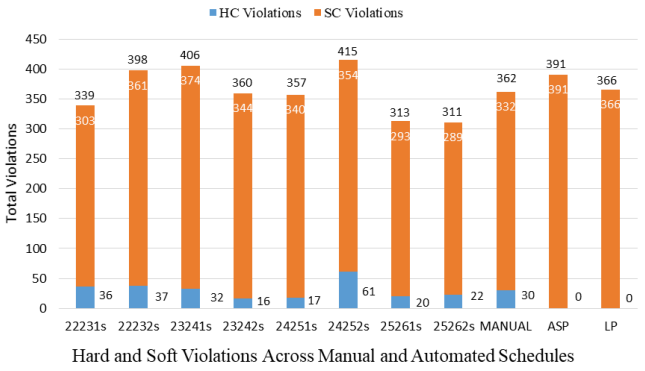


Fig. 11. Hard and Soft Violations Across Manual and Automated Schedules

Based on the results as seen in **Figure 11**, both ASP and LP consistently achieved zero hard constraint violations across all runs, while the manual schedules exhibited multiple hard constraint violations. The consistent difference demonstrates that the automated methods satisfied all required hard constraints, indicating a better performance and higher schedule quality compared with manually prepared schedules.

For the soft constraints, manual schedules occasionally produced fewer violations than automated schedules. This was particularly evident when compared with ASP, which does not fully optimize soft constraint violations, while LP performs more effectively in managing these constraints. However, this slight advantage in soft constraints minimization came at the cost of repeated hard constraints violations, which significantly reduced the overall validity of the manually created schedules. Moreover, the number of soft constraint violations may also be influenced by servicing subjects that were not included within the scope of the study, as professors handling additional courses could have preferences that were not captured in the model. Consequently, the automated solvers ensured valid schedules by prioritizing hard constraint satisfaction over soft preferences. Furthermore, the differences in soft violations between the two methods were relatively minimal, highlighting the competitiveness of solver-generated schedules in preference optimization.

To further examine the recurring conflict patterns found in the previous schedules, the hard and soft constraint violations from all eight manually created schedules were analyzed in greater detail.

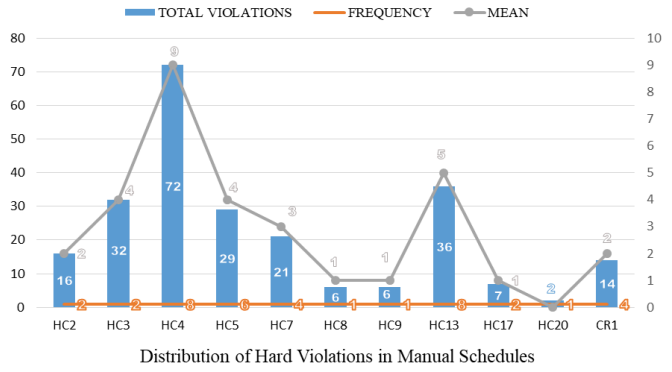


Fig. 12. Distribution of Hard Violations in Manual Schedules

Figure 12 presents the total number of hard violations across all the manual schedules, including their mean scores and how often they occur across all eight previous semesters. These violations were particularly evident in HC2 (No Professor Overlap), HC3 (No Room Overlap), HC4 (Room Capacity), HC5 (Room Type), HC7 (Professor Consecutive Limit), HC8 (No Three Consecutive Classes), HC9 (No Two Consecutive Laboratory Classes), HC13 (Professor Forbidden Timeslots), HC17 (Standard Day Pairing), HC20 (No Two Consecutive 3-Hour Classes), and CR1 (Course Allowed Rooms). Among these, HC4 recorded the highest number of violations and frequency of occurrence, followed by HC13.

The chart indicates persistent overlap-related and room-related violations, particularly those involving professor unavailability and room suitability. These issues become increasingly difficult to manage as departments and course offerings expand. In contrast, ASP and LP successfully eliminated these recurring conflicts, which demonstrated a higher quality in creating schedules.

Moreover, the breakdown of the hard violation totals for each of the eight prior manual schedules is shown separately in the figure below to further illustrate the distribution of these hard constraint violations.

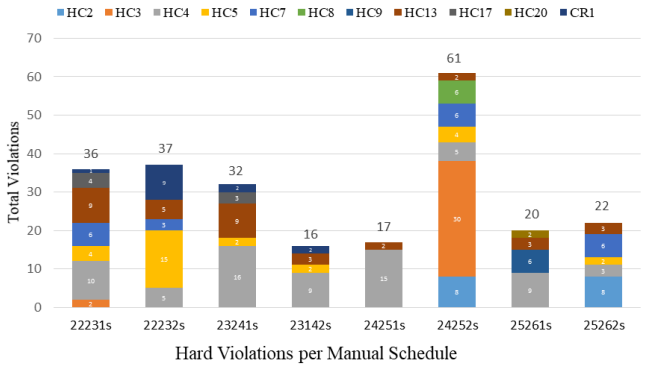


Fig. 13. Hard Violations per Manual Schedule

Figure 13 illustrates that hard constraint violations occurred consistently in all manually created schedules, varying in number by semester. This indicates ongoing challenges in meeting critical scheduling rules within a manual approach. From this, the repeated presence of hard violations across multiple semesters further strengthens the advantage of the automated solvers, which consistently maintained zero hard violations in every generated schedule.

For the soft constraints, **Figure 14** below demonstrates the total number of soft violation counts across all manual schedules, including their mean score, and frequency of occurrence.

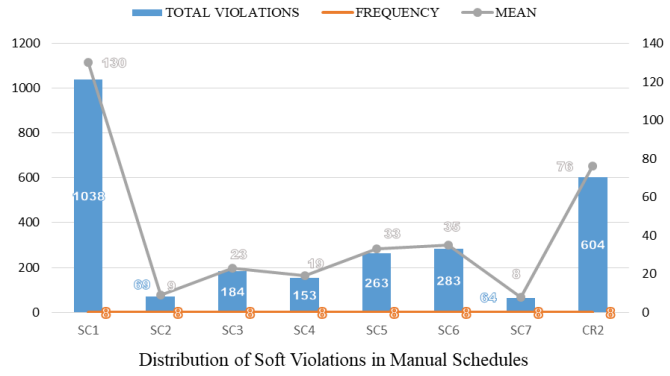


Fig. 14. Distribution of Soft Violations in Manual Schedules

It can be observed in the figure that almost all soft constraints are present in all schedules. Among the soft constraints, SC1 (Professor Preferred Timeslots) was the most frequently violated preference. Notably, as discussed earlier, SC1 was also the most common soft violation in both ASP and LP. This indicates that the professor's preferred timeslots remain the key optimization trade-off in automated scheduling.

This analysis is significant as it shows that both automated solvers encountered the same preference conflicts as manual scheduling while still avoiding hard violations. However, there were also instances where manual schedules exhibited high soft violations, particularly 22232s, 23241s, and 24252s, further proving that the slight average advantage was not consistently maintained in the manual scheduling process.

Likewise, the breakdown of the soft violation totals for each manual schedule are shown separately in the figure below.

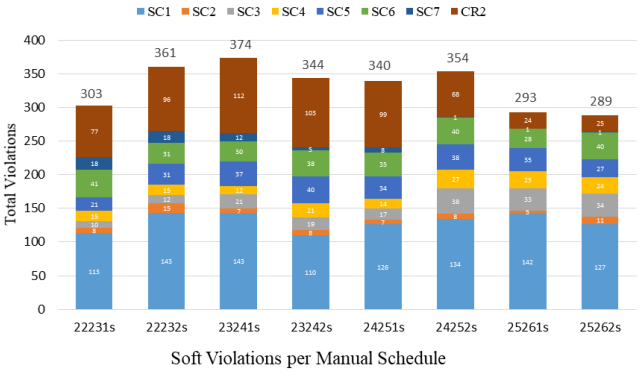


Fig. 15. Soft Violations per Manual Schedules

Figure 15 indicates that manually created schedules varied in performance, with some achieving lower soft violations but some with higher violations. In contrast, automated solvers, particularly LP, showed more stable results, ensuring a balance between fulfilling hard constraints and minimizing soft violations.

Overall, the findings show that while the manual approach may occasionally reduce certain preference-based violations, it does so by compromising critical violations of hard constraints. In contrast, both automated solvers provide a more reliable optimization strategy, particularly LP, which satisfies all hard constraints while maintaining only minimal and acceptable soft preference trade-offs.

b) Penalty Points

This section provides a penalty point comparison, which further strengthens the observed advantage of the automated scheduling methods over the manual approach in creating a schedule. The figure below shows the penalty points for all manual schedules, along with their mean value, and compares them with the results from ASP and LP.

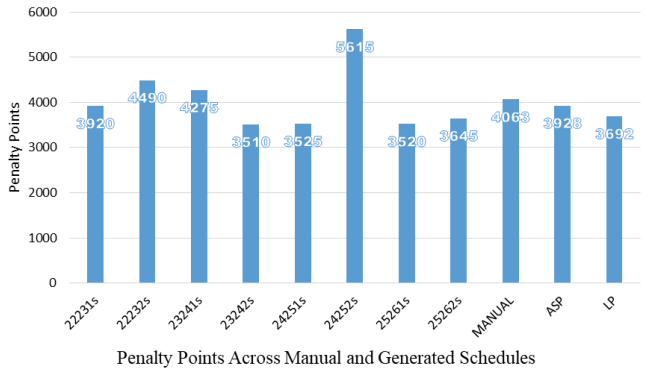


Fig. 16. Penalty Points Across Manual and Generated Schedules

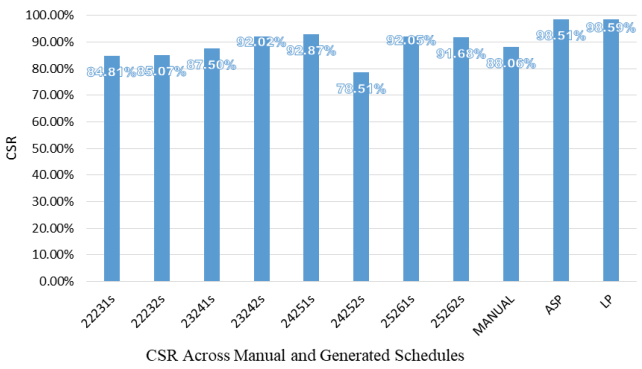


Fig. 17. CSR Across Manual and Generated Schedules

As presented in **Figure 16**, the manual schedules obtained the highest average penalty points among all evaluated methods, while both ASP and LP produced substantially lower mean penalty scores across ten runs each, with LP being the lowest. Since the penalty model assigns larger weights to hard constraint violations, the repeated hard conflicts present in the manual schedules significantly increased the total accumulated penalty points. This directly reflects the lower scheduling quality of the manual method.

In accordance with that, the lower penalty points of both ASP and LP result from their ability to maintain zero hard constraint violations. Their remaining penalties came mainly from soft preference conflicts, particularly related to professors' preferred timeslots. In contrast, manual schedules faced penalties from both soft preferences and significant hard violations, leading to a significant total penalty difference.

From this, the results indicated that the automated scheduling system method was able to produce a higher quality solution in creating a schedule in CSEA, with LP particularly evident in gaining the lowest penalty points. Obtaining lower penalty points reflects schedules that align closely with the ideal objectives of this study. That being said, this supported the study's aim to evaluate whether ASP and LP enhance scheduling quality compared to manual methods, with findings showing that both solvers created a more optimized schedule than the traditional approaches.

c) Constraint Satisfaction Rate

In this section, the CSR results confirmed the comparative findings and analysis from the violation counts and the penalty points. The figure below shows the CSR for all manual schedules, along with their mean value, and compares them with the results from ASP and LP.

As shown in **Figure 17**, ASP and LP achieved high CSR values, with LP being the highest, indicating that these two methods consistently satisfy nearly all scheduling rules. On the other hand, manual schedules had lower satisfaction rates.

The high CSR values of ASP and LP were mainly due to their complete compliance with all the hard constraints provided. While some soft constraints like preferred timeslots may be only partially met, the overall satisfaction remains significantly high due to the minimal impact of these trade-offs. In comparison, the lower CSR values in manual schedules result from numerous hard conflicts and inconsistent compliance with soft preferences.

This result highlighted the practical reliability of the automated scheduling system for departmental use in CSEA. Both methods effectively satisfy complex academic scheduling constraints that involve faculty, rooms, and sections, as well as the provided constraints. Moreover, the consistently high CSR values of ASP and LP indicated their ability to manage complex problem analysis while maintaining the quality of the generated schedules. Thus, the findings strongly supported that the automated methods offered an overall solution quality and constraint compliance over manual scheduling.

V. EVALUATION

This section presents the results of the end-user evaluation conducted among the department chairpersons from CSEA following the deployment and testing of the scheduling system application. The evaluation included three (3) current department chairpersons who used the system in a simulated academic scheduling context. Feedback was collected via simple YES/NO questions, structured SUS and Likert scale instruments to assess usability, functionality, and overall satisfaction, providing insights into the system's effectiveness and potential for departmental scheduling support and assistance.

A. Task-Based Evaluation

In this section, specific tasks were assigned to the department chairpersons to guide them in navigating the application as they attempted to generate schedules for their respective departments. These are the following:

1. Review or upload the required scheduling data
2. Generate a schedule using the solver
3. Review the generated schedule
4. Check hard constraints, soft constraints, and penalty points
5. Compare generated schedule outputs
6. Select the preferred schedule for department use

The figures below present a detailed breakdown of the responses provided by the three (3) department chairpersons regarding their experience in testing the scheduling system application. Specifically, the data highlight their level of task completion, the occurrence of any errors encountered, and the extent to which assistance was required during the process of generating a schedule.

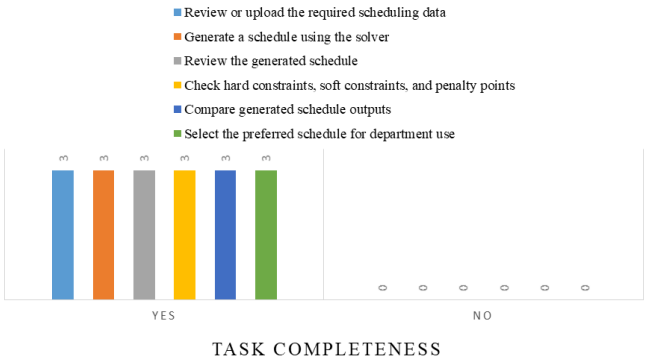


Fig. 18. Task Completeness in Using the Application

Figure 18 indicates that all three department chairpersons were able to successfully complete the tasks assigned to them. This, therefore, demonstrates that the application effectively supports users in accomplishing the required scheduling functions.

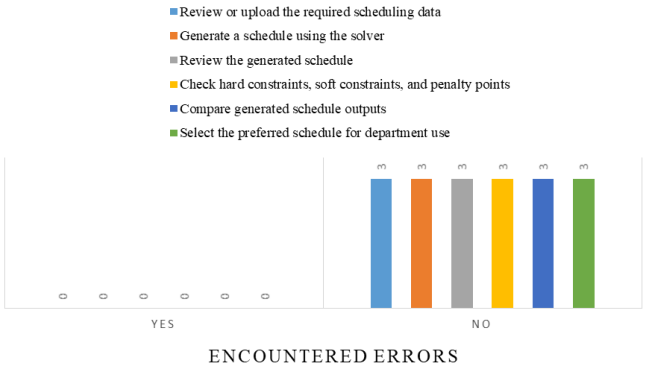


Fig. 19. Encountered Errors in Using the Application

Figure 19 shows that none of the chairpersons encountered any errors while using the system during the execution of the given tasks. This also suggests that the application is stable and functions as intended under the tested conditions and required tasks.

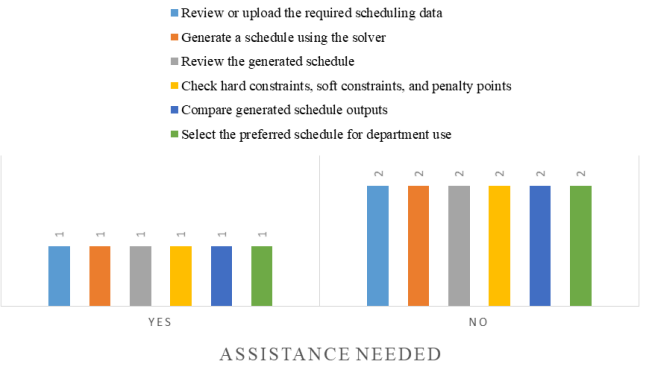


Fig. 20. Assistance Needed in using the Application

In terms of user independence, **Figure 20** demonstrates that only one out of the three chairpersons required assistance while using the application, while the remaining two were able to navigate and complete the tasks without further assistance.

Overall, the results generally support that the system is reliable and effective for scheduling by department chairpersons. While assistance was minimally needed, one user's need for assistance suggests areas for improvement in interface clarity and user guidance.

B. System Usability Scale

The System Usability Scale (SUS) was administered to three department chairpersons to evaluate the perceived usability of the developed system. The evaluation instrument used for this section consisted of the following default questions:

1. I think that I would like to use this application frequently.
2. I found the application unnecessarily complex.
3. I thought the application was easy to use.
4. I think that I would need the support of a technical person to be able to use this application.
5. I found the various functions in this application were well integrated.
6. I thought there was too much inconsistency in this application.
7. I would imagine that most people would learn to use this application very quickly.
8. I found the application very cumbersome to use.
9. I felt very confident using this application.
10. I needed to learn a lot of things before I could get going with this application.

From that, the table below shows the breakdown of answers from the three (3) chairpersons.

TABLE XII. SYSTEM USABILITY SCALE RESPONSES

QUESTION NUMBER	SCALE ANSWERS		
	Chairperson #1	**Chairperson #2**	**Chairperson #3**
#1	5	5	3
#2	2	5	4
#3	5	5	4
#4	2	1	2
#5	5	5	3
#6	1	1	3
#7	5	5	3
#8	2	1	2
#9	5	5	4
#10	2	1	3
SUS	**90**	**90**	**57.5**
	Average SUS		**79.17**
	Percentile		**85-89% (A-)**

As shown in the results in **Table XII**, the system received a SUS score of 90 from Chairpersons #1 and #2, and a score of 57.5 from Chairperson #3. The average SUS score across the three evaluators was calculated at 79.17, which falls well above the set average usability criterion of 68[18]. This indicates that the system is generally perceived as highly usable by the target users, which in this case are the department chairpersons within CSEA. In terms of percentile ranking, the obtained average score corresponds to the 85-89% range, which is equivalent to a grade of A-. This implies a high average score of usability for the system application.

For a deeper observation, the individual responses reveal strong agreement on the positive statements (odd-numbered questions), with both Chairpersons #1 and #2, who are consistently rating most items with a score of 5 (strongly agree), while Chairperson #3 provided slightly more moderate ratings than the two. That being said, the responses were largely low for the negative statements (even-numbered questions), suggesting that there were few observed usability problems with the deployed application. However, Chairperson #3 had comparatively higher scores on a few negative items (which were reverse-scored during computation), which somewhat reduced the overall SUS of the system. Despite this minor variation, the results still show that the system passed the usability test with an above-average rating, indicating strong user satisfaction, usability, and overall acceptability among the department chairpersons. These results demonstrate how well the system's functionality and interface satisfy the needs of its intended users.

C. Solver and Schedule Quality Evaluation

In this section, an evaluation for the solver and schedule quality was conducted to assess the effectiveness of the developed scheduling tool in generating optimized schedules within the CSEA. The evaluation focused on the quality, accuracy, and acceptability of the schedules produced by the solver based on predefined academic scheduling constraints and institutional requirements.

The evaluation instrument used for this section consisted of the following questions:

1. The generated schedule was acceptable for department use.
2. The generated schedule was practical and realistic to implement.
3. The system was able to produce schedules with minimal conflicts.
4. The assigned rooms, timeslots, and faculty assignments were appropriate.
5. The hard constraints and soft constraints were reflected properly in the generated output.
6. The penalty points and schedule violations were easy to understand.
7. The comparison of generated schedules helped me decide which schedule was better.
8. The system can reduce the time and effort needed for manual scheduling.
9. I trust the generated output enough to use it as a basis for actual scheduling.
10. The system is useful for assisting department chairs in schedule preparation.

From that, the table below shows the breakdown of answers from the three (3) chairpersons.

TABLE XIII. SOLVER AND SCHEDULE QUALITY EVALUATION

Question Number	SCALE ANSWERS			RESULTS	
	Chairperson #1	Chairperson #2	Chairperson #3	Mean Score	Final R
#1	5	5	4	4.67	5
#2	5	5	3	4.33	5
#3	5	5	4	4.67	5
#4	5	5	3	4.33	5
#5	5	5	4	4.67	5
#6	5	5	4	4.67	5
#7	5	5	4	4.67	5
#8	5	5	3	4.33	5
#9	5	5	3	4.33	5
#10	5	5	4	4.67	5

Based on the results shown in **Table XIII**, the three chairpersons who used the system gave consistently high ratings across all ten statements in the evaluation. Items 1, 3, 5, 6, 7, and 10 obtained a mean score of 4.67, while items 2, 4, 8, and 9 received a mean score of 4.33. Despite the slight differences in the individual responses, all statements were interpreted with a final rating of 5, which indicates a very high level of acceptability and satisfaction with the quality of the generated schedules from both methods, ASP and LP, used. The final rating was based on the weighted mean interpretation scale from Chapter 2 (see section 2.1.23 for a more detailed explanation).

As seen from the results, the developed scheduling system effectively assists department chairpersons in creating schedules, from its received high ratings. However, it can be noted that items 2, 4, 8, and 9 received lower mean scores of 4.33, indicating minor reservations from the end-users about the practicality of generated schedules, resource assignment, and trust in the output for scheduling. This may be from a preference for manual validation due to unique departmental needs or last-minute adjustments of the schedules. Despite recognizing the system's effectiveness in scheduling, the lower scores suggest that human oversight remains crucial for special cases that automation might not address effectively.

Overall, the results from the evaluation confirm that the system can substantially reduce the time and effort required in manual scheduling while improving the quality and reliability of the generated schedules. The ability of both solvers in the two methods to generate optimum outputs that meet department chairpersons' demands and institutional scheduling criteria is further supported by the consistently high results from the department chairpersons.

D. Chairperson Recommendation

The figure below shows how likely the department chairperson is to recommend the scheduling system to other department chairpersons.

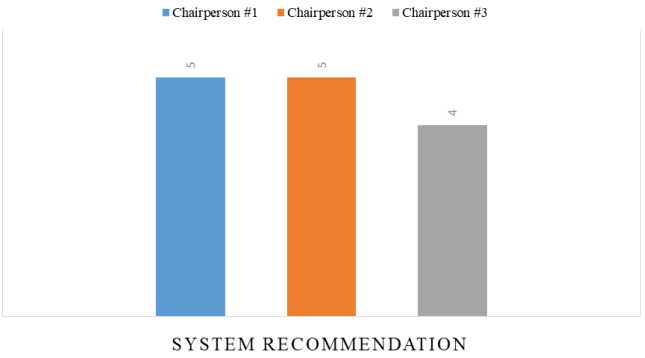


Fig. 21. Chairperson System Recommendation

Figure 21 shows the results of both Chairperson #1 and Chairperson #2 recommending the system with a score of 5, while Chairperson #3 assigned a score of 4. Following the point system in the Likert-Scale, this yields a mean score of 4.67, which falls within the range of 4.21 to 5.00, interpreted as "Strongly Agree" with a score of 5. This indicates that respondents exhibit a very high willingness to recommend the system to other department chairpersons, with consistently high ratings that also reflect a strong satisfaction in performance, usability, and effectiveness for academic scheduling tasks.

VI. CONCLUSION

This study designed and implemented a class scheduling system utilizing Answer Set Programming (ASP) and Linear Programming (LP) within the scope of the College of Science, Engineering, and Architecture (CSEA) at Ateneo de Naga University. The system was developed to address the inefficiencies and limitations associated with manual scheduling by generating conflict-free and optimized class timetables based on institutional constraints.

This study organized scheduling data into a structured database using MySQL enabling data management. In addition, institutional rules and preferences were systematically encoded as hard and soft constraints for both solvers, ensuring that generated schedules adhered to essential policies while allowing for optimization. Moreover, a web-based application was developed using FastAPI for the backend and React.js with Tailwind CSS and TypeScript for the frontend, providing an intuitive graphical user interface that allows department chairpersons to input data, generate schedules, and review results within a centralized system.

The findings of the study indicated that both ASP and LP were effective approaches in generating feasible schedules within the defined study scope, as both methods consistently satisfied all hard constraints. This ensured the validity of the generated timetables and demonstrated that the system was capable of addressing essential scheduling requirements.

In terms of computational performance, ASP exhibited significantly faster solver times, producing schedules within a

shorter duration compared to LP. This suggested that ASP was more suitable for scenarios requiring rapid schedule generation. However, this advantage was accompanied by a trade-off, as ASP resulted in a higher number of soft constraint violations, indicating lower optimization in schedule quality. Conversely, LP demonstrated superior performance in optimizing schedule quality. It consistently generated schedules with fewer soft constraint violations and lower penalty points, resulting in a higher overall constraint satisfaction rate. These results suggested that LP was more effective in accommodating faculty preferences and enhancing the overall usability of the schedules, albeit at the cost of longer computation time.

Furthermore, in comparison with manual scheduling, the automated system demonstrated improvements in minimizing constraint violations, reducing scheduling conflicts and overlaps, and improving overall scheduling quality within the implemented evaluation framework. However, this comparison should be interpreted within the limits of the study, since manual upload evaluations were limited to constraints that could be fairly verified from the uploaded schedules and the available supporting data.

The evaluation results also indicated high levels of user satisfaction and acceptability, suggesting that the system was practical and suitable for real-world academic scheduling applications. However, the evaluation was limited to the three participating department chairpersons, specifically the Electronics and Computer Engineering Chairperson, the Mathematics Chairperson, and the Natural Sciences Chairperson within CSEA. Moreover, the generated schedules remain draft outputs that are still subject to final review and approval by the department chairpersons.

The study concluded that the automated class scheduling system can serve as a practical decision-support tool for class scheduling in CSEA within the study's defined scope. Under the tested conditions, ASP offered faster schedule generation, while LP provided better soft-constraint optimization and overall schedule quality. The findings could further benefit from the use of more complete datasets.

References

[1] F. M. Salem et al., "IEEE Xplore Full-Text PDF," IEEE, 2025. Accessed: Nov. 18, 2025. [Online]. https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9499056

[2] S. Larabi-Marie-Sainte, R. Jan, A. Al-Matouq, and S. Alabduhadi, "The impact of timetable on student's absences and performance," PLOS ONE, vol. 16, no. 6, p. e0253256, Jun. 2021, doi: https://doi.org/10.1371/journal.pone.0253256.

[3] P. T. Nsulangi, Werneld Egno Ngongi, Mansour Rafii Likamba, Omary Bakari Sarehe, and Miraji Abdallah Mkwande, "A Comparative Analysis of Manual and Automatic Timetabling Approaches for Resource Utilisation in Tertiary Higher Learning Institution," International Journal of Computer Science and Mobile Computing, vol. 13, no. 12, pp. 65–76, Dec. 2024, doi: https://doi.org/10.47760/ijcsmc.2024.v13i12.007.

[4] K. Sushil and Saini, "Issue 3 www.jetir.org (ISSN-2349-5162)," JETIR2303854 Journal of Emerging Technologies and Innovative Research, vol. 10, 2023, https://www.jetir.org/papers/JETIR2303854.pdf

[5] Ahmed Redha Mahlous and Houssam Mahlous, "Student timetabling genetic algorithm accounting for student preferences," vol. 9, pp. e1200–e1200, Feb. 2023, doi: https://doi.org/10.7717/peerj-cs.1200.

[6] Wang, C., "A Course Scheduling System Based on an Improved Genetic Algorithm for Complex Scenarios," NHSJS Reports, vol. 2025, pp. 1-10, Oct. 31, 2025. https://nhsjs.com/wp-content/uploads/2025/11/A-Course-Scheduling-System-Based-on-an-Improved-Genetic-Algorithm-for-Complex-Scenarios.pdf

[7] M. Khadivi et al., "Deep reinforcement learning for machine scheduling: Methodology, the state-of-the-art, and future directions," Computers & Industrial Engineering, vol. 200, p. 110856, Jan. 2025, doi: https://doi.org/10.1016/j.cie.2025.110856

[8] Miguel, C. Sousa, and S. Vieira, "A Systematic Review on Reinforcement Learning for Industrial Combinatorial Optimization Problems," Applied Sciences, vol. 15, no. 3, pp. 1211–1211, Jan. 2025, doi: https://doi.org/10.3390/app15031211.

[9] V. Lifschitz, "What Is Answer Set Programming?" Available: https://www.cs.utexas.edu/~vl/papers/wiasp.pdf

[10] E. Pellicer, J. Yepes, V. Singh, M. Humphrey, and A. Singh, "Resilient Structures and Sustainable Construction Edited by A LINEAR PROGRAMMING APPLICATION AND SOLUTION FOR MINIMIZING CLASS SCHEDULING CONFLICTS." Available: https://www.isec-society.org/ISEC_PRESS/ISEC_09/pdf/EPE-15.pdf

[11] De Lima, A., De Sousa, B., Cardeal, D., Sato, J., Salvador, L., Geh, R., & Bazaluk, B. (2023). LUNCH: an Answer Set Programming System for Course Scheduling. Anais do XX Encontro Nacional de Inteligência Artificial e Computacional (ENIAC 2023).

[12] C. Dohrmann, U. Lucke, T. Schaub, and S. Schellhorn, "AI-Based Tool for Curriculum-Based Course Timetabling at the University of Potsdam," EPiC Series in Computing, vol. 105, pp. 175–188

[13] J. R. Gryzen, "Optimizing the Peter Kiewit Institute course schedule using Answer set programming," DigitalCommons@UNO. https://digitalcommons.unomaha.edu/university_honors_program/350/

[14] K. Bushman, Using Visualization and Integer Linear Programming for University Class Scheduling, M.S. thesis, Dept. of Computer Science, Univ. of Pittsburgh, Pittsburgh, PA, USA, Jul. 2021

[15] R. Hoshino and J. Albers, "Cohort-Based Timetabling with Integer Linear Programming," Khoury College of Computer Sciences, Northeastern University, Vancouver BC, Canada. [Online]. Available: https://www.khoury.northeastern.edu/home/rhoshino/papers/nu1.pdf

[16] "Optimization of University Course Scheduling Problem using Particle Swarm Optimization with Selective Search," Science Direct, [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0957417419301393

[17] J. Sauro PhD, "Measuring Usability with the System Usability Scale (SUS) – MeasuringU." https://measuringu.com/sus

[18] "View of a statistical analysis of System Usability scale (SUS) evaluations in online learning platform." https://journal-isi.org/index.php/isi/article/view/750/373

[19] E. Nyutu, W. Cobern, and B. Pleasants, "ArticlePDF Available Correlational Study of Student Perceptions of their Undergraduate Laboratory Environment with respect to Gender and Major," ResearchGate, Dec. 2020, [Online]. Available: https://www.researchgate.net/publication/348329285_Correlational_Study_of_Student_Perceptions_of_their_Undergraduate_Laboratory_Environment_with_respect_to_Gender_and_Major